软件培训 · 第三次

电机闭环控制

先弄清开环、闭环与反馈，认识电机和编码器；

再把位置式与增量式 PID 摆在一起，看它们各自怎么用在速度环上。

原理优先　两种 PID 对照　含烧录与上电

CH32V307 · 直流有刷编码器电机 · 上位机 VOFA+ · 现场演示验收

01 原理
02 器件
03 出力
04 实现
05 PID
06 验收

控制就是：拿实际值和目标比，再决定下一步做什么

接水、骑车、调电机，做的都是同一件事。

01 目标
想要多少
接满一杯水；电机每 5 ms 转 50 个计数。
`r = 50`

02 测量
现在多少
眼睛看水位；编码器数这一拍的增量。
`y = 43`

03 比较
差多少
还差 7 个计数。这个差值，是后面所有计算唯一的输入。
`e = r − y = 7`

04 执行
多给一点
水快满了就关小；电机慢了就把占空比加上去。
`u += Δu`

换个说法

控制不是“把输出设成某个值”，而是“每个周期看一眼结果，再决定下一步改多少”。前者是开环，后者是闭环。

最容易翻车的一步

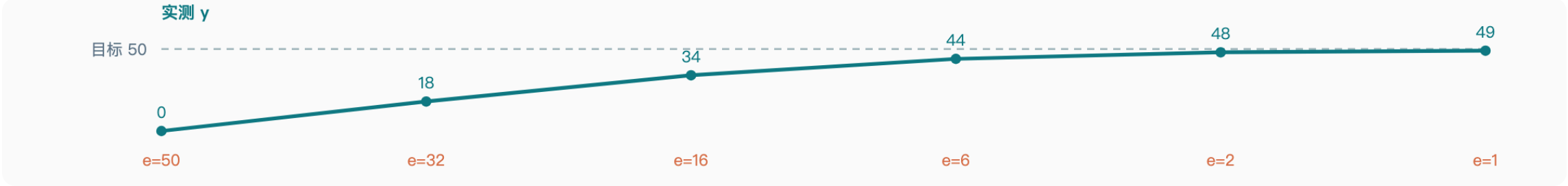
目标用“50 个脉冲”，反馈却换成 rpm 去减。量纲不一致，后面怎么调都不对。

记住这个问句

现在差了这么多，下一步输出该变多少？PID 就是这句话的一个具体答案。

三件事都要成立

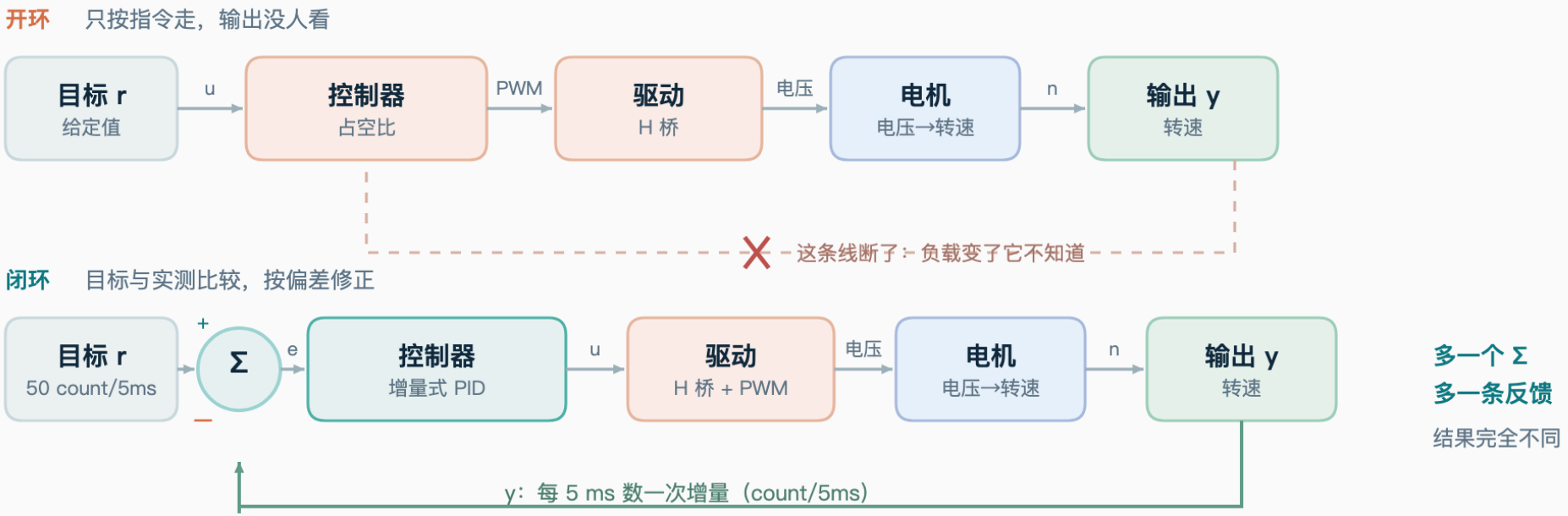
测量要准、比较要同量纲、执行要够快。缺一个，闭环就调不出来。



每一拍看一次结果：目标不动，实测追上来，偏差越来越小。

开环与闭环：差的就是那条反馈线

同一个目标、同一台电机，多一个 Σ 和一条反馈，系统就不一样了。



多一个 Σ
多一条反馈
结果完全不同

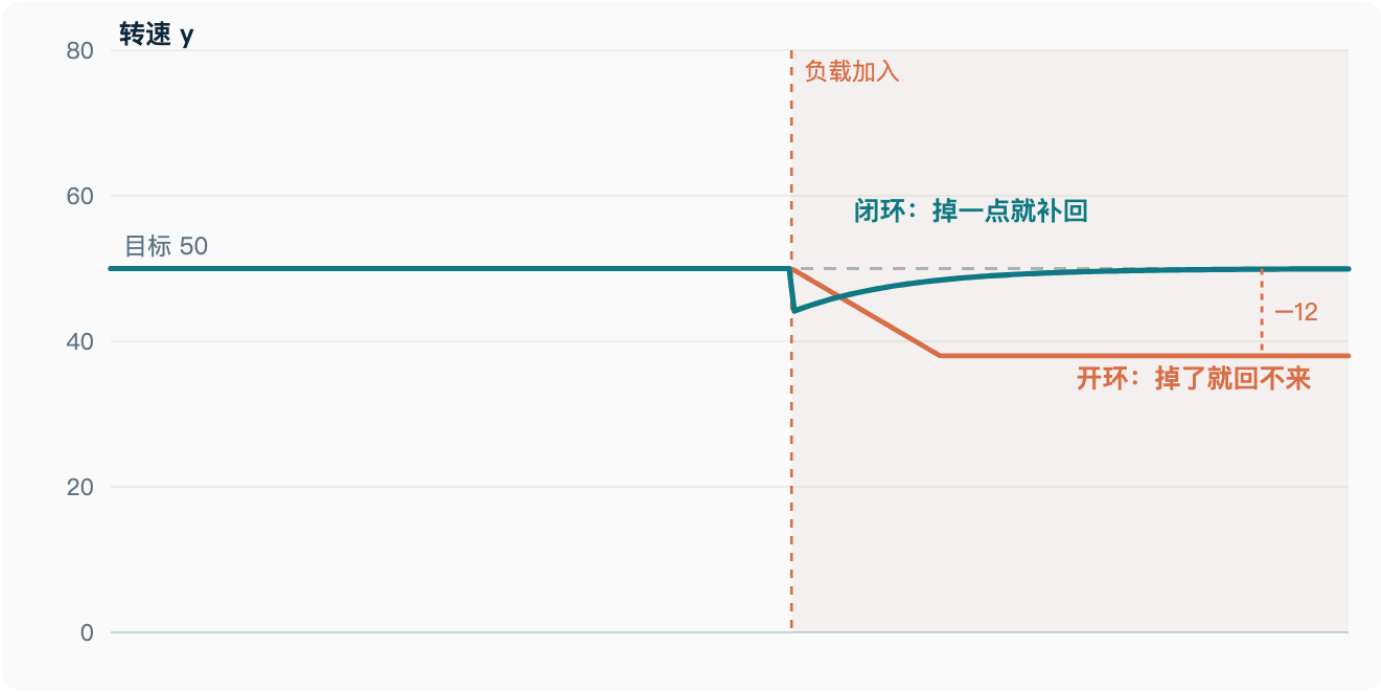
开环：输出只是被看见，不参与决定。**闭环**：每 5 ms 把实测值送回比较，差多少补多少。

控制与计算　出力与驱动　电机　反馈与编码器

只多了一条线，代价在后面
传感器、固定周期、参数，都要跟着来。

闭环为什么能抗干扰

干扰一出现，闭环马上知道“不对劲”；开环永远不知道。



同一条负载扰动：开环掉下去就回不来，闭环掉一点就被补回来。

负载变大

上坡、压到地面、轮子阻力变大。转速一掉，偏差变大，占空比自己加上去。

电池电压下降

电量低、大电流时压降。同样的占空比出力变小，闭环会多给一点。

摩擦与温度变化

轴承磨损、温度升高。这类慢变化，靠积分项一点点补上。

但闭环不是万能的

目标超出电机能力（比如空载都到不了 80 个计数）时，怎么调都上不去。调参看三个数：掉多少、多久回来、稳态波动多大。

闭环不是免费的

系统更“聪明”，也更难调、更容易振。

开环 给多少就是多少

- 结构：一段赋值代码，给个固定占空比
- 调参：不用调
- 抗干扰：没有
- 风险：几乎不会失控

闭环 看结果再决定

- 结构：固定周期 + 反馈 + 控制器
- 调参：按对象调 1–3 个参数
- 抗干扰：有，但受输出能力限制
- 风险：参数不当会振荡、顶满、反转

多出来的四样东西

传感器：编码器和接线；**固定周期**：每拍读、算、写，不能忽快忽慢； **参数**：$K_p/K_i/K_d$ 要按对象调；**风险**：参数不合适会振，越调越差。

什么时候可以不用闭环

负载基本不变、对速度要求不高（风扇、小水泵），开环给个电压就够； 车轮要按目标速度走、还要抗坡道和摩擦变化，就必须闭环。

问题 1

负载会变吗

坡道、摩擦、电池压降——只要会变，开环就稳不住。

→

问题 2

速度要稳住吗

差一点能接受就开环；要求跟着目标走，就得闭环。

→

问题 3

传感器和周期有吗

没有编码器、给不出固定周期，闭环免谈。

先分清四类电机

换向由谁完成、有没有内置反馈，决定了它好不好控制。

	机械换向 电刷 + 换向器	电子换相 驱动电路按序通电
无内置反馈 开环也能转	**直流有刷** 本次 两根线加电压就转，一路 H 桥就够。有火花、寿命有限，常见于小车与玩具。 驱动：H 桥 · 外挂编码器即可速度闭环	**步进电机** 按节拍轮流励磁，不用反馈也能定位；高速力矩衰减快，可能丢步。常见于 3D 打印机。 驱动：专用步进驱动器 · 闭环需另加编码器
有内置反馈 自己就在闭环	**舵机** 有刷电机 + 减速箱 + 位置反馈，给角度指令就到位。转向、云台都用它。 驱动：给脉宽 · 内部已经是位置闭环	**直流无刷** 转子永磁、定子三相，电子换相。效率高、寿命长，但必须先知道转子位置。风机、无人机常见。 驱动：三相驱动器 · 能闭环，换相逻辑复杂

为什么这次选有刷

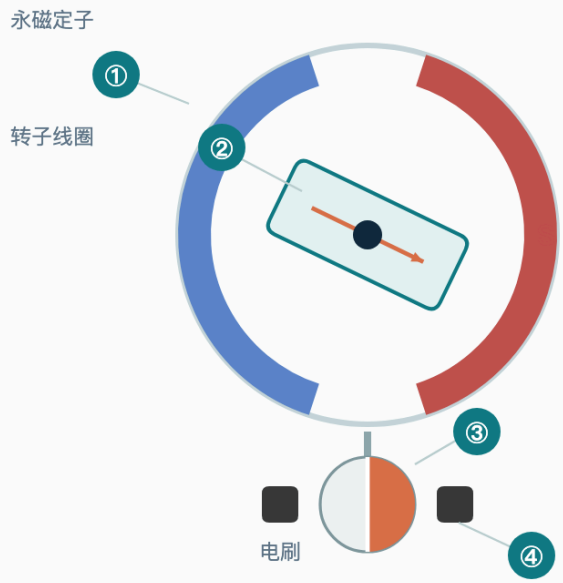
一路 PWM + 一个 H 桥 + 外挂编码器，就能把“反馈、测速、闭环”讲完整，是硬件最少的一种。

CH32V307	有刷 + 编码器	H 桥驱动	12 V 电池
主控	被控对象	出力	供电

直流有刷电机里面有什么

四个部件，各干一件事；右边三步说明“为什么每半圈要换一次电流方向”。

剖面



换向三步



电流在 90° 处反向，转矩方向保持不变

剖面里 ①–④ 对应下排四张卡；三步图里电流方向在 90° 处翻转，转矩方向始终不变。

① 定子磁铁

提供恒定磁场。退磁 → 力矩明显变小。

② 转子线圈

通电后受安培力产生转矩。烧断 → 完全不动。

③ 换向器

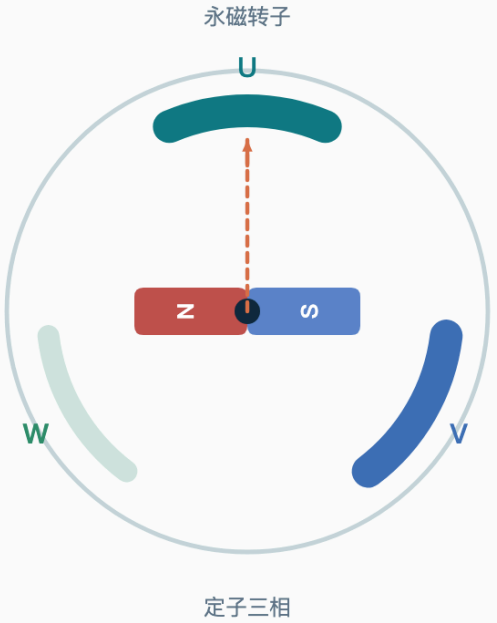
每半圈交换一次接触片。磨损 → 抖动、异响。

④ 电刷

把电流送到转动的换向器上。磨损 → 火花大、接触不良。

无刷电机：没有电刷，换相交给电路

转子是永磁体，定子是三相绕组；驱动器按顺序通电，磁场一步步转，转子被拖着走。



六步换相

步	通电	磁场
1	U → V	0°
2	U → W	60°
3	V → W	120°
4	V → U	180°
5	W → U	240°
6	W → V	300°

右侧六步会自动循环；亮的两个绕组就是当前通电的两相，箭头是合成的磁场方向。

六步换相

每 60° 电角度换一次绕组组合，磁场“跳”六次转一圈。任何时刻只通电两相，第三相悬空。

为什么复杂

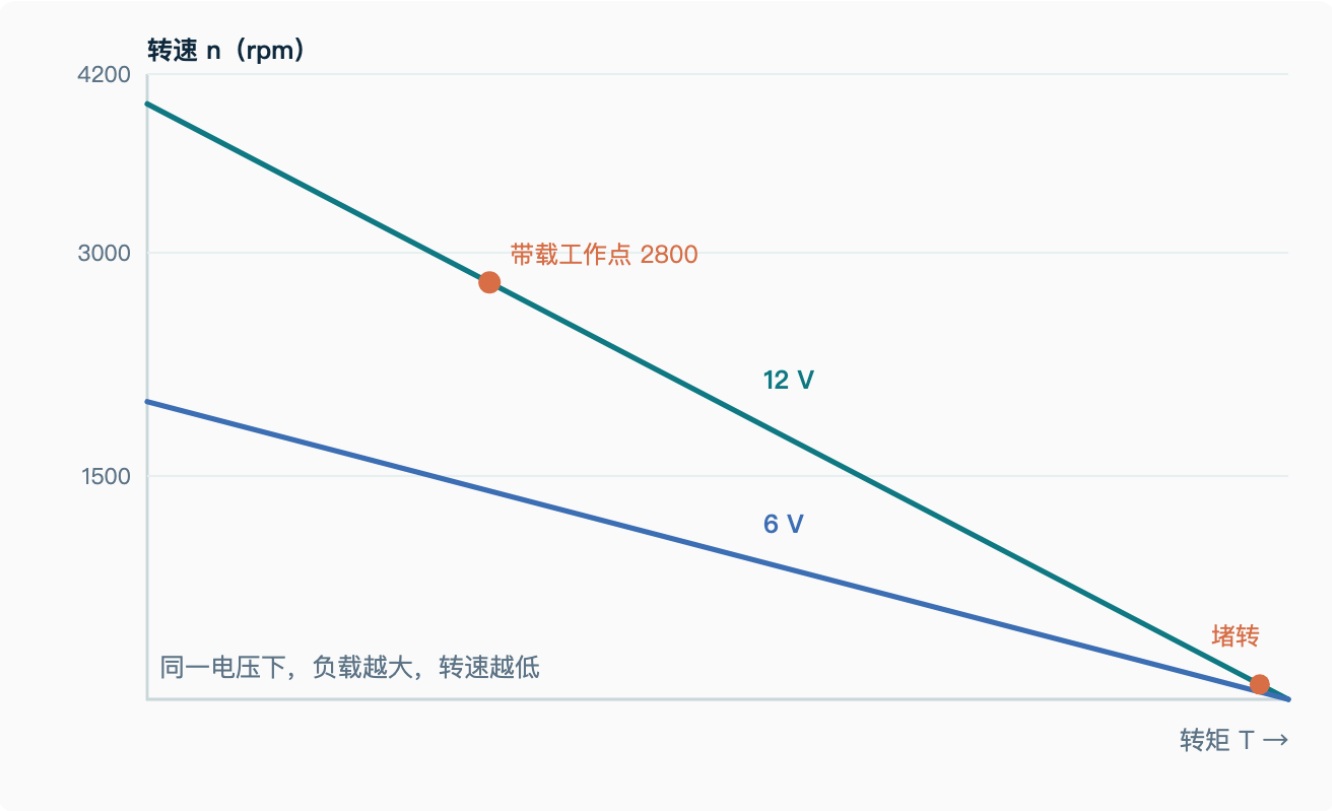
必须先知道转子在哪（霍尔 / 编码器 / 无感反电动势），否则不知道该通哪两相。

和有刷的对照

都是“让电流跟着转子转”；有刷靠机械，两根线就行；无刷靠电路，必须配驱动和位置检测。

电压定转速，负载定电流

右图是一条机械特性：横轴转矩、纵轴转速，线上任何一点都能稳定工作。



同一电压下，负载越大，工作点沿斜线往右下走，转速越低。

空载：$n_0 \approx K_v \cdot U$

带载：$n \approx K_v \cdot (U - I \cdot R)$

转矩：$T \approx K_t \cdot I$

量	含义	单位
U	电机两端电压	V
I	流过的电流	A
R	电枢内阻	Ω
K_v	转速常数（每伏多少转）	rpm/V

看得出来的规律

电压升高，整条线往上平移；转矩加大，工作点往右下走，转速就降了。

负载一大，为什么转速就掉

电机转动时自己会产生“反抗电压”，转速越高，反抗越强。

1. 转子切割磁感线，产生反电动势 $E = K_e \cdot n$，方向和电源相反。
2. 真正推动电流的是差值：$I = (U - E) / R$。
3. 负载加大 → 要更大转矩 → 要更大电流 → **I 变大**。
4. U 不变、I 变大，只能 **E 变小** → 转速 n 下降。

结论

只要电压不变，负载一变转速就变。想让转速不受负载影响，就得在负载变化时主动改 U——这正是闭环要做的事。

算一个例子

U = 6 V，R = 2 Ω，K_e = 0.01 V/rpm。

空载　I ≈ 0.1 A　**580 rpm**

带载　I = 1.0 A　**400 rpm**

同样 6 V，掉了 180 rpm。

闭环怎么补

发现只有 400，就把占空比从 50% 提到 70%，U 变 8.4 V，E 回到 5.8 V，转速回到 580。

$$U = E + I \cdot R = K_e \cdot n + I \cdot R \;\rightarrow\; n = (U - I \cdot R) / K_e$$

堵转为什么最危险

n=0 时 E=0，电流只剩 U/R，是正常转动的十几倍——最容易烧电机和驱动。

反电动势还有别的用

无感无刷就是靠它判断转子位置；这里只要记住：它随转速升高而变大。

编码器：把“转了多少”变成数字

增量式有两种接口——正交 A/B 和带方向（PUL+DIR）；绝对式上电就知道在哪。测速只需要增量式。

正交 A/B 本次

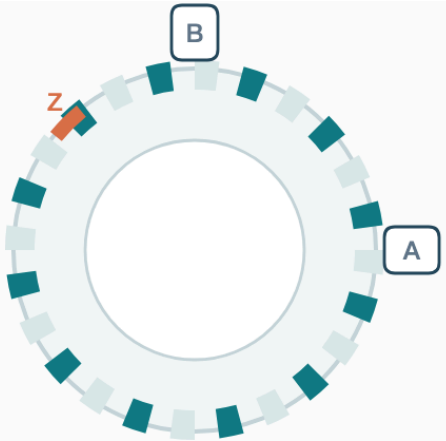
两路方波差 90°，靠相位先后判向；4 倍频，分辨率最高；要两路输入。

带方向编码器

一路脉冲 PUL + 一路方向电平 DIR；DIR 直接表示正反；1 倍频，接线少一根。

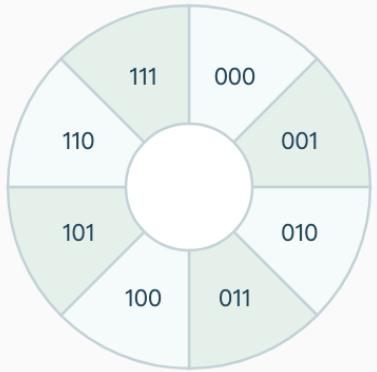
绝对式

每个角度位置一个唯一编码，上电就知道在哪；接口复杂、成本高。



增量式：一圈 N 个缝

A/B 错开 1/4 缝距，Z 每圈一次



绝对式：每格一个编号

上电就知道自己在哪一格

左：一圈刻缝 + 错开 1/4 缝距的 A/B 两路，另有一个每圈一次的 Z。右：每个扇区自带编号，掉电也知道在哪。

Z 相干什么

每转只出一个脉冲，用来“对零”：过一次 Z，计数就有了确定参考。

选型看三个数

每转脉冲数（分辨率）、最高响应频率、输出类型（开漏 / 推挽 / 差分）。

正交 A/B：两路相差 90°，才能判方向

拖滑块或点「让它转」，看游标扫过四个状态；波形会自己滚动。



轴的角度 **0°**

方向：正转

让它转

A / B	累计计数	判定
10	0	正转（A 领先）

① 90° 从哪来

A、B 两组透光缝错开 1/4 个缝距。

② 四个状态

正转依次 10 → 11 → 01 → 00，每态占 1/4 周期。

③ 判方向

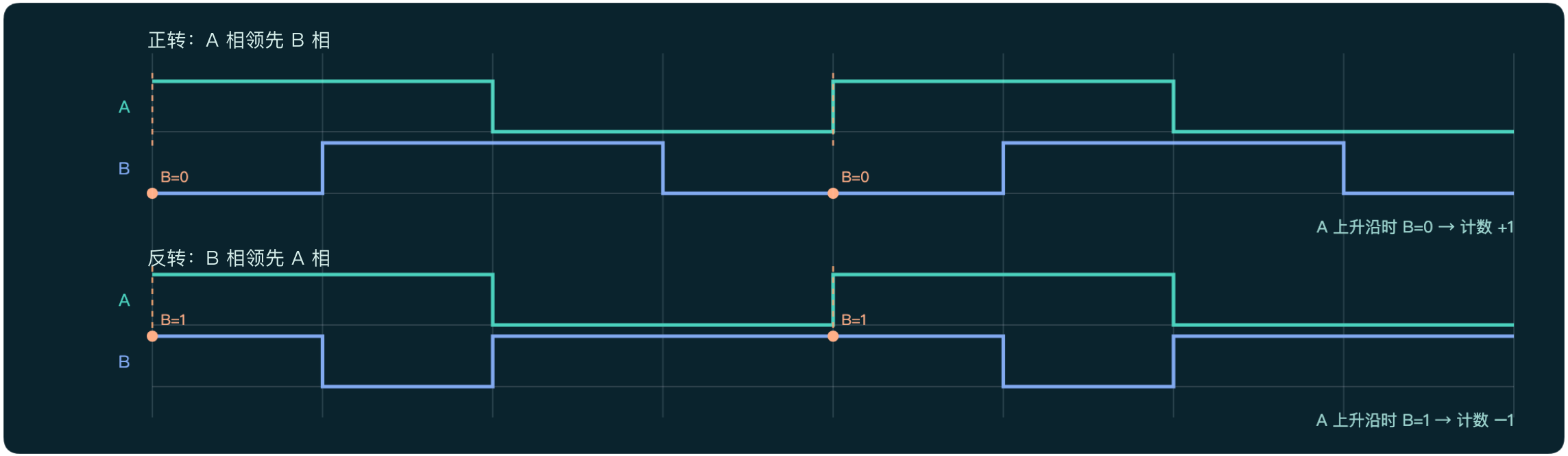
A 上升沿时 B=0 → 正转；B=1 → 反转。

④ 单路不够

只接 A 能数“转了多少”，但不知道往哪转。

同样是增量式，判向有两种做法

正交 A/B 看两路方波的先后；带方向编码器只看一根 DIR 电平。这次板子是正交。



看 A 的上升沿：B 还是低电平 → A 领先 → 正转；B 已经是高电平 → 反转。

正交 A/B

两路方波，看相位先后判向；4 倍频，分辨率最高；A、B 都进定时器，硬件解码。

带方向编码器 PUL+DIR

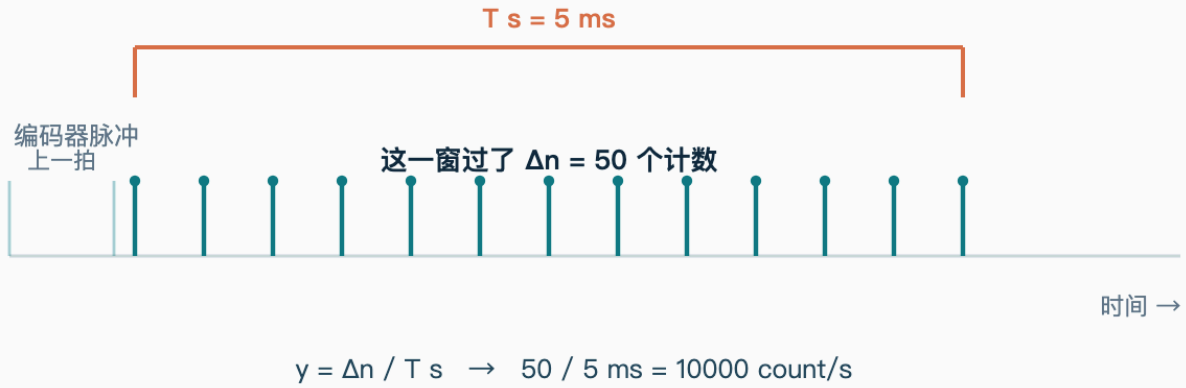
一路脉冲计数，一路电平直接给方向；1 倍频，接线少一根，但多占一个 GPIO。

这次用哪种

板子是正交 A/B，定时器编码器模式；软件只读计数差，不自己判相位。

测速：固定窗口内数增量

编码器给的是位移，配上时间才是速度。窗口越短越灵敏，计数越少越粗糙。



每 5 ms 开一次窗，数窗口里过了多少个脉冲；这个数就是这一拍的速度。

$$y = \Delta n / T_s \quad (\text{count} / s)$$

$$\text{rpm} = 60 \times \Delta n / (C \times T_s)$$

Δn（5 ms 内）	C = 1040	计数率
10	115 rpm	2000 count/s
25	288 rpm	5000 count/s
50（默认）	**577 rpm**	**10000 count/s**
100	1154 rpm	20000 count/s

量化误差

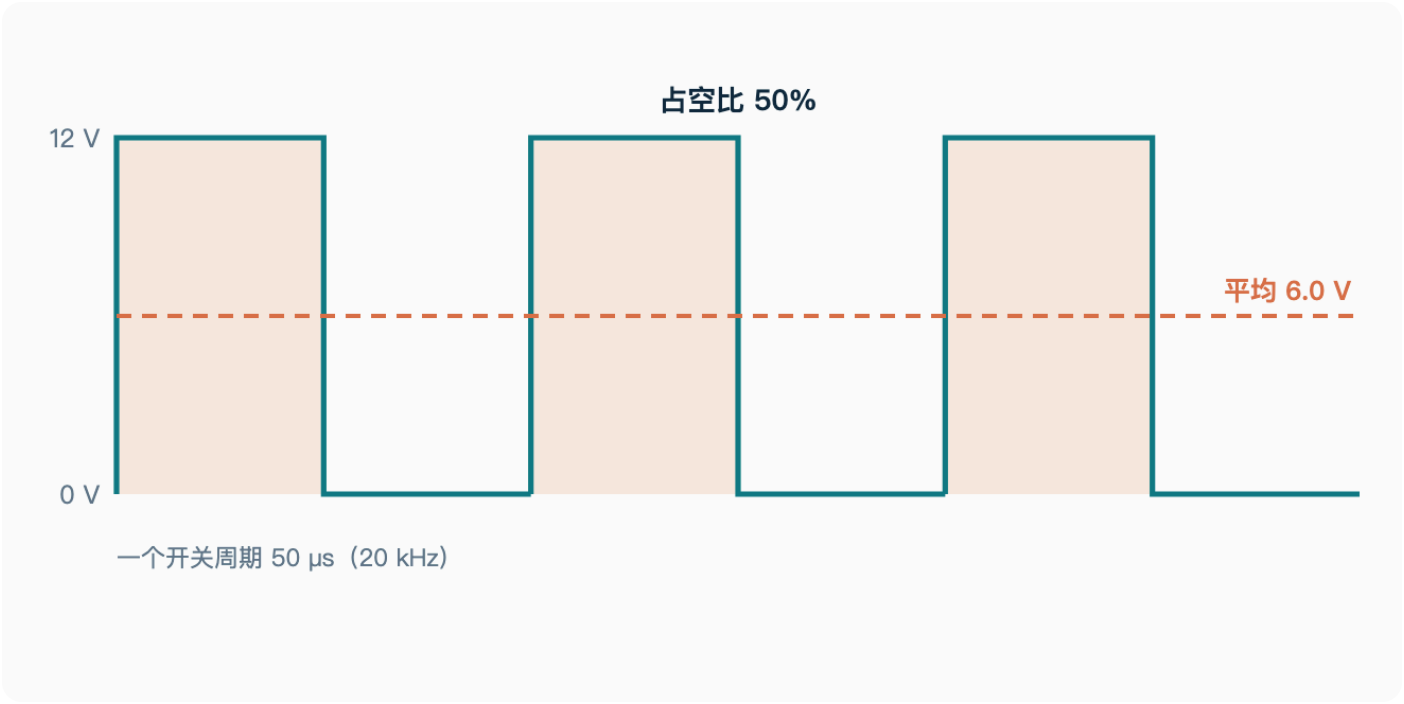
计数是整数，目标 50 时 1 个计数就是 2%，读数在 49/50/51 之间跳是正常的。

换算时只用 count

目标和反馈都用“每 5 ms 多少计数”，就不必关心电机型号、减速比和线数。C 不确定时，手转一圈数计数，以实测为准。

单片机怎么“给”出一个中间电压

IO 只会输出高或低；开关足够快，电机只感受到平均值。拖滑块试试。



橙色面积的平均高度就是平均电压；占空比一变，电机两端看到的电压就变。

占空比 **50%**

6.0 V
平均电压（12 V 供电）

2000
空载转速 rpm（估算）

两个频率别混

开关频率 20 kHz 决定电流平不平滑、有没有啸叫；**控制频率** 200 Hz 决定多久修正一次。

100% 也不是全速

还有电池压降、驱动压降、电机内阻，实际总比理想值低一点。

为什么是 20 kHz

高于人耳听觉，电机不啸叫；周期 50 μs，电流纹波也小。

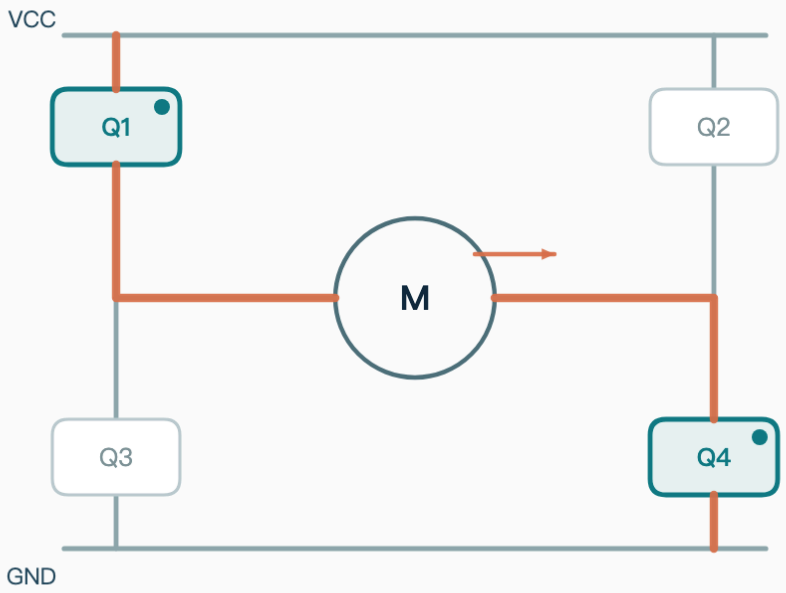
太低会怎样

电流断续、电机抖动、有可闻噪声；频率太高则开关损耗上升。

方向由 H 桥决定，大小由占空比决定

单片机引脚只有毫安级电流，电机要安培级；H 桥负责功率放大和换向。

正转：Q1 + Q4 导通



点右边的状态，看电流从哪两条腿走；同一桥臂上下一起导通就是电源短路。

正转 Q1+Q4 | 反转 Q2+Q3 | 刹车 下桥全通 | 滑行 全关断

一条硬规矩：不能直通

同一桥臂上下两个开关同时导通 = 电源短路。换向前先关断，必要时加**死区**（几十 ns 到几 μs）。

PWM 加在哪

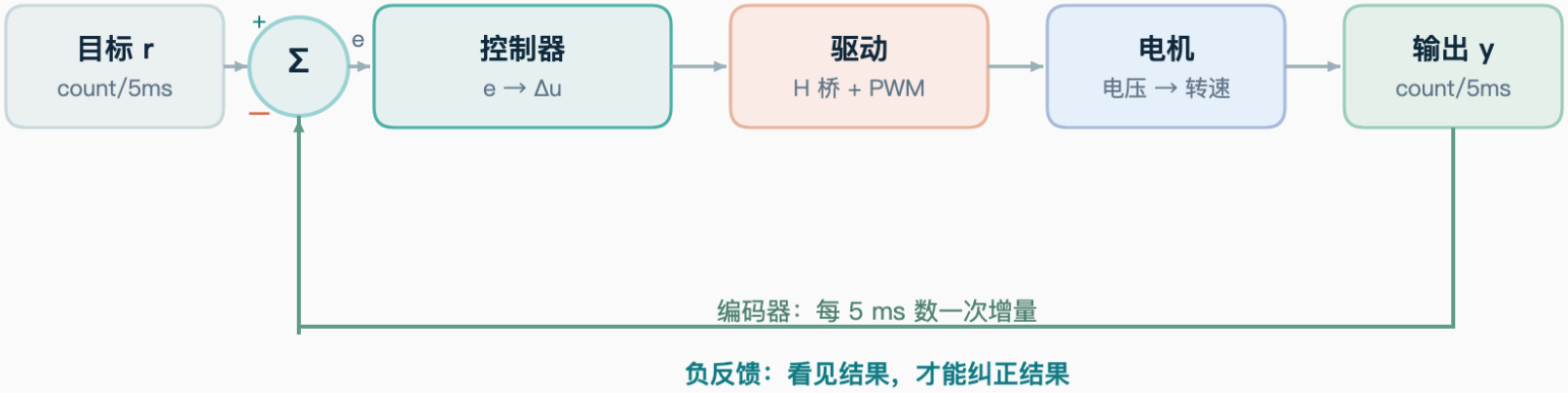
常见做法是“一路 PWM + 另一路常低”，或两路互补 PWM；方向由导通组合决定，大小由占空比决定。

选驱动看三个数

耐压 ≥ 电源 2 倍余量、持续电流 ≥ 堵转电流、导通电阻决定发热。

把零件连成一条闭环

每个方框都对得上工程里的一个函数或一个外设；信号名和单位要一致。



负反馈：看见结果，才能纠正结果

目标 r 与反馈 y 都用 count/5 ms；控制器输出 u 是占空比，单位 %，限幅 ±100。

量纲要一致

拿计数去减 rpm 或占空比，怎么调都不对。

符号要正确

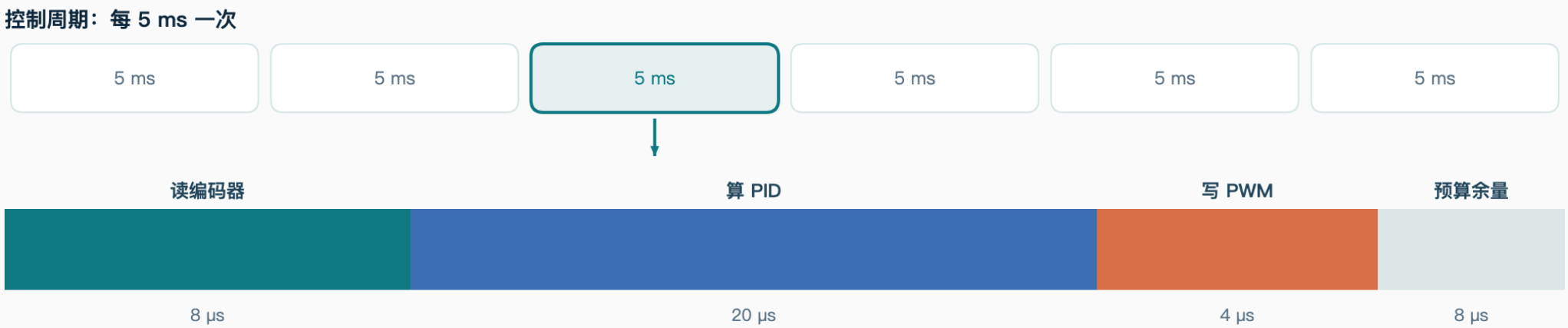
先确认“PWM 增大 → 计数增大”；反了就是正反馈，几拍顶满。

控制器是什么

拿偏差算输出的那个部件：输入 e，输出 u。换电机只改参数，结构不变。

每 5 ms 做四件事

读、算、写必须在同一个节拍里完成；跨节拍错位，控制器就拿旧数据做新决定。



一个周期 5 ms，四步加起来约 40 µs，只占 0.8%；剩下时间留给主循环刷屏和打印。

① 读编码器

取走本周期增量 Δn，并清零计数器。

② 算转速

y = Δn，单位 count/5 ms。

③ 算偏差

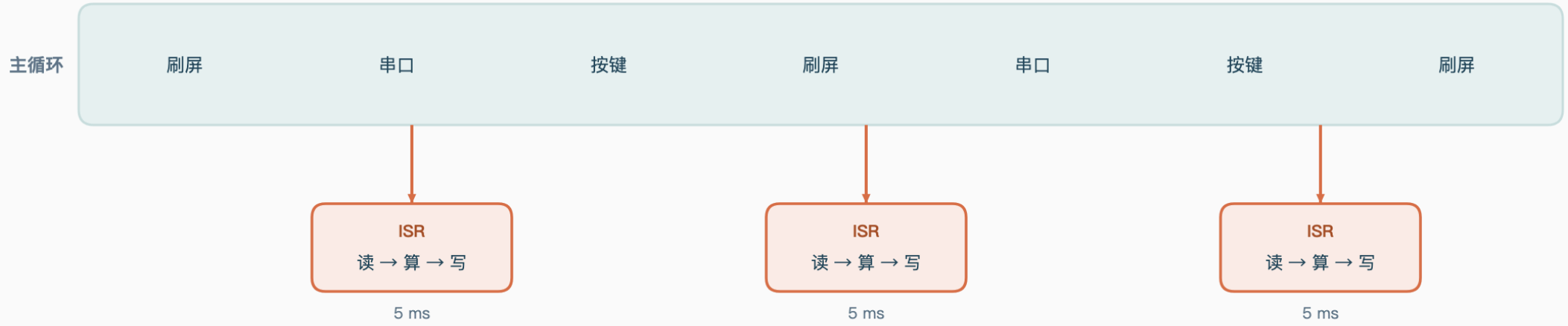
e = 目标 − y，交给 PID 算 Δu。

④ 更新 PWM

u += Δu，限幅后写比较寄存器，并把 u 一起发给 VOFA。

让周期不受主循环影响

主循环里用 while 延时算出来的周期不确定；一打印、一刷屏，周期就变了。



定时器每 5 ms 打断一次主循环，进 ISR 把“读、算、写”做完，几十微秒后返回。

ISR 里只做

读计数 → 算 PID → 写 PWM，几十微秒结束。

ISR 里不要

printf、刷屏、动态内存、while 等待。

volatile

中断里改的变量要加，否则可能被优化掉。

原子性

主循环读多字节变量可能被打断，读两次或关中断保护。

main.c 只负责调度，逻辑分模块

把所有代码堆进 main.c 不算完成；一个模块只做一件事，换器件才不牵一发动全身。

工程目录（示意）

```
project/
├── code/
│   ├── motor.c / .h        方向 + PWM + 限幅
│   ├── encoder.c / .h      正交计数、读增量
│   ├── pid.c / .h          增量式 PID（纯数学）
│   └── vofa.c / .h         串口上报（可选）
└── user/src/
    ├── main.c              初始化 + 启动定时器 + 打印
    └── isr.c               TIM 中断 → 调 pid_step()
```

对外接口只有四个

```
motor_set(int u)
encoder_read_delta(void)
pid_update(float target, float y)
vofa_send(…)
```

三条规矩

main.c 里不出现 PID 公式与寄存器操作；模块内部变量用 static；头文件只放函数原型与宏。

为什么要拆

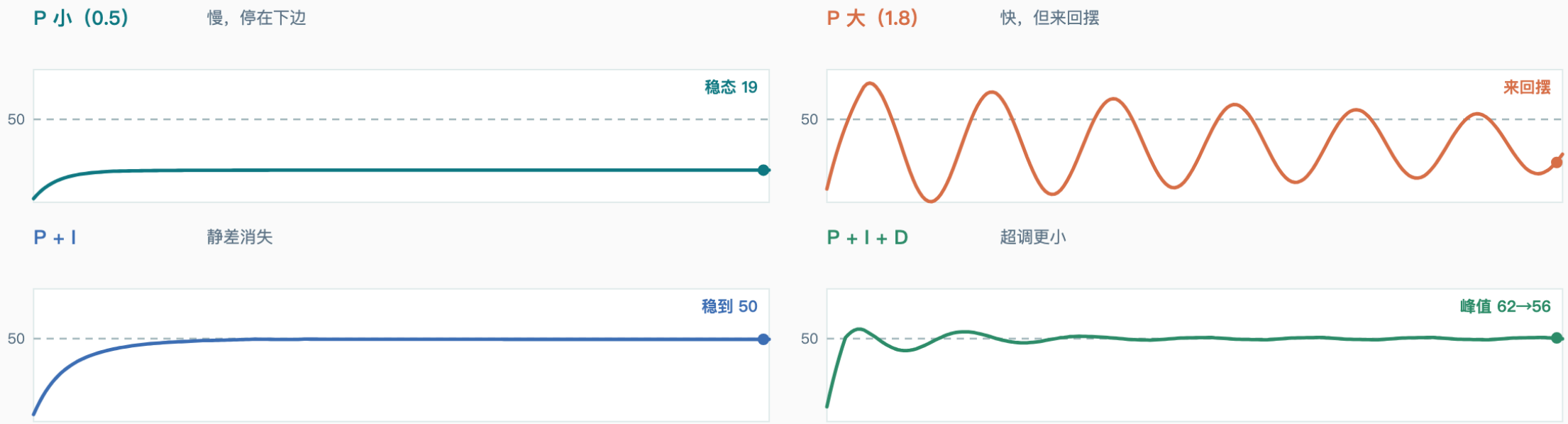
换电机只动 motor.c，换编码器只动 encoder.c，调参只动 pid.c——其他文件不用重读。

验收会这样问

“这个文件负责什么？改目标值要动哪一行？PID 公式在哪个文件？”——答不上来，说明还是堆在一个文件里。

P 看现在，I 看历史，D 看趋势

同一台电机、同一个目标，只改参数，看四条曲线差在哪。



P 小：慢、停在下边；P 大：快但来回摆；加 I：静差消失；再加 D：峰值压低。

P：差多少补多少

输出 = K_p × 偏差。响应最快，但偏差为 0 时出力也为 0，所以纯 P 一定有静差。

I：把历史累加起来

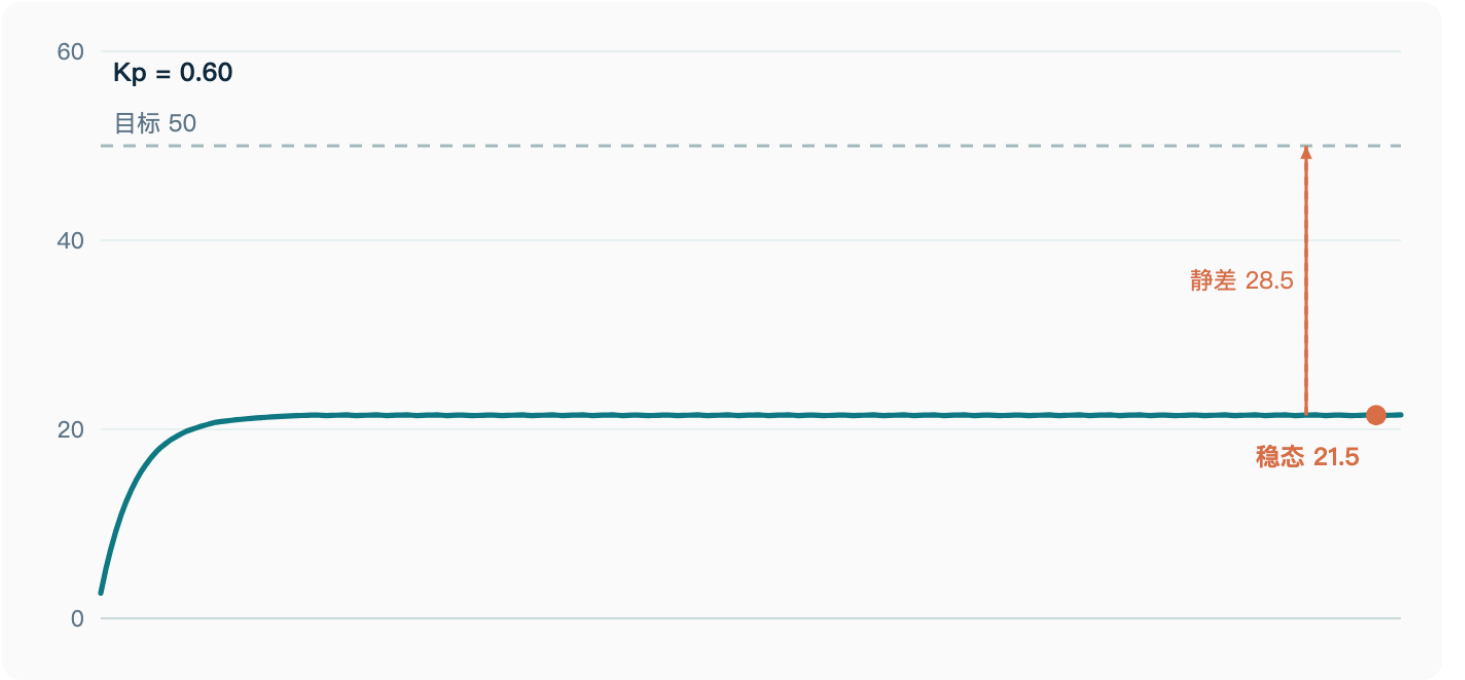
误差消失后累加值仍在，正好提供维持转速的出力；太强会超调、来回摆。

D：看变化趋势

速度冲得太快时给反向修正，压超调；差分会放大计数噪声，速度环常常不加。

只有 P，就一定会差一点

不是参数没调好，是结构决定的：误差为 0，P 的出力也为 0。拖 K_p 看稳态点怎么挪。



目标线不动，K_p 再大，稳态点也只能逼近、贴不上。

稳态：$y = K_v K_p r / (1 + K_v K_p)$

静差：$e = r / (1 + K_v K_p)$

K_v 是电机增益（这台约 1.25），不是 K_p

K_p **0.60**

21.5
稳态 y（count/5 ms）

28.5
静差（count/5 ms）

结论

加大 K_p 只能把静差压小，消不掉；要消静差必须加 I。

每次只算“变化多少”，再加回输出

位置式相邻两拍相减，就得到这一拍的增量；只依赖最近三次偏差。

$$\Delta u = K_p(e_k - e_{k-1}) + K_i \cdot e_k + K_d(e_k - 2e_{k-1} + e_{k-2})$$

$$u = u + \Delta u$$

单位

三个系数都折算过 T_s，统一为“% / (count/5 ms)”，代码里直接用。

上电第一拍

e1、e2、u 必须清零，否则第一次的微分项是随机值。

先累加再限幅

限幅后的值写回 u，下一拍才从受限值继续加。

5 ms 中断里的速度环

复制

```
/* 量纲统一：count/5ms 与 % */
static float e1, e2, u;

void speed_loop_5ms(void)
{
    int   n = encoder_read_delta();
    float y = (float)n;
    float e = 50.0f - y;    /* 默认目标 */

    float du = KP*(e - e1)
             + KI*e
             + KD*(e - 2.0f*e1 + e2);

    u += du;
    if (u >  100.0f) u =  100.0f;
    if (u < -100.0f) u = -100.0f;

    motor_set((int)u);
    e2 = e1;
    e1 = e;
}
```

位置式：算出“这一脚该给多大”

输出就是控制量本身，误差多大油门就给多大；积分是一个要自己看住的状态。

$$u = K_p \cdot e + K_i \cdot \Sigma e + K_d \cdot (e - e_1)$$

积分要限幅

Σe 会一直涨。输出顶到 100 之后积分还在累，松手就先冲一下——这就是积分饱和，必须单独夹住。

重启要清状态

integ、e1 不清零，第一拍就会带着上一次的积分跑。

改参数会跳

u 是直接输出，K_p 一改，工作点立刻变。要平滑，得做无扰切换。

一句话

它最像人开车：看着速度表，脚直接踩到某个深度。

同样的 5 ms 速度环

复制

```
/* 位置式：输出全量，积分显式 */
static float integ, e1;

void speed_loop_5ms(void)
{
    int   n = encoder_read_delta();
    float y = (float)n;
    float e = 50.0f - y;

    integ += e;                      /* 积分显式累加 */
    if (integ >  2000.0f) integ =  2000.0f;  /* 抗饱和 */
    if (integ < -2000.0f) integ = -2000.0f;

    float u = KP*e + KI*integ + KD*(e - e1);
    if (u >  100.0f) u =  100.0f;
    if (u < -100.0f) u = -100.0f;

    motor_set((int)u);
    e1 = e;
}
```

数学上是一回事，调起来是两回事

同一组系数算出的 u 一样；但先拧哪个旋钮、盯哪条线，是两套习惯。

位置式

$u = K_p e + K_i \Sigma e + K_d(e - e_1)$

直接算全量，积分是显式状态。

相邻两拍
相减
→

增量式

$\Delta u = K_p(e - e_1) + K_i e + K_d(e - 2e_1 + e_2)$

只算这一拍的变化，再累加回 u。

关键结论

同一组 $K_p/K_i/K_d$，两种写法算出的 u 一模一样；但调参手感差很远——先拧谁、看哪条线，是两套习惯。

限幅

位置式夹住 u，integ 还在涨；增量式夹 u 就是夹状态，天然抗饱和。

重启

位置式要清 integ、e1；增量式要清 u、e1、e2。

噪声

位置式 D 是一阶差分；增量式 D 是二阶差分，同样的 K_d 更吃噪声。速度环干脆不加 D。

起手

位置式先给 P、盯 u；增量式先给 I、盯 Δu。先拧谁不一样。

所以

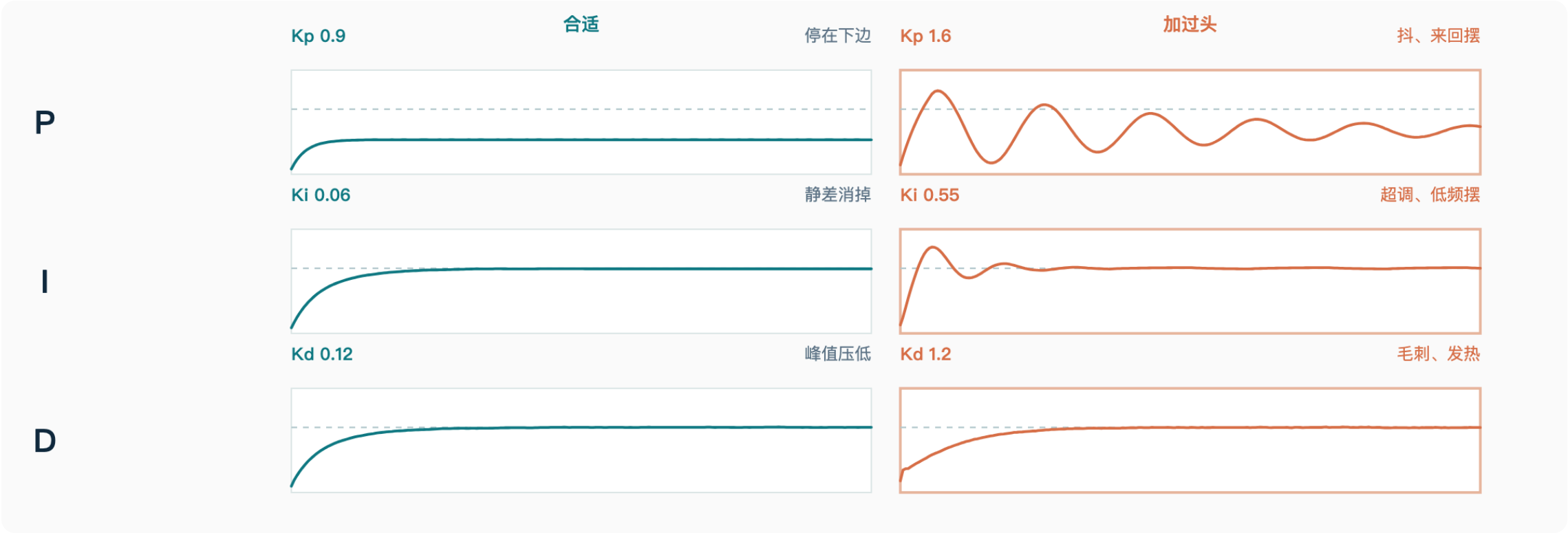
数学等价，不等于调参一样；也别把“增量式更先进”讲成结论。

自己验证一次

把同一组系数分别填进两种写法，曲线应该完全重合；不重合，先查限幅写回和上电初值。

每个旋钮，都有“合适”和“过头”

同一台电机，只改一个量；左边是加到刚好，右边是加多了。



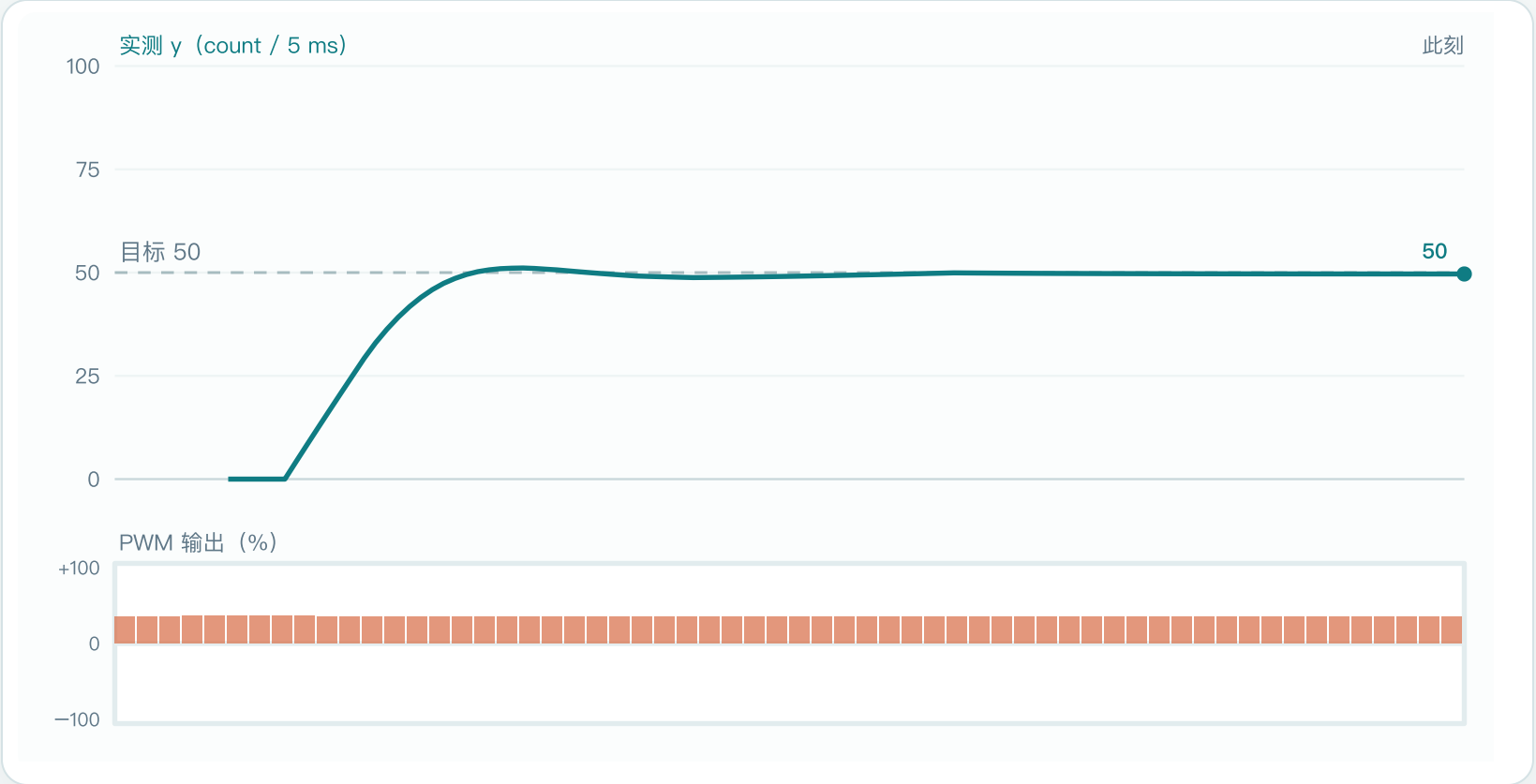
橙框那一列是“加过头”：P 过头抖、I 过头摆、D 过头毛。

一句话

P 管硬度，I 管静差，D 管趋势；三个都是过犹不及。加多了先退半步，再谈别的。

改一个参数，看两条曲线

上方是转速（虚线为目标），下方橙色是 PWM；按住“受阻”相当于捏住电机轴。



增量式 | 位置式

- [ ] 计数噪声 ×3
- [x] 保留上一轮（灰线）

K_p 0.90

K_i 0.06

K_d 0.00

目标 50

限幅 ±100%

重新开始 | 暂停 | 按住受阻

目标 +10 | 目标 −10

k	e	Δu	u	实测 y	峰值	超调	静差
110	0	0.00	39.7	50	51	2%	0.3

先调谁，取决于你看的是 u 还是 Δu

两条路都能到同一组系数，区别是起手先动哪个旋钮、看哪条曲线。

位置式：P → I → D

1. **P 加到“微微超调”**
   振荡要少，别加到刚好不差
2. **加 I**
   一定要小，慢慢加；大了会低频摆
3. **必要时加 D**
   压超调；I 可以不加
4. **每次只改一个量**
   改完看三条曲线

增量式：I → P，D 基本不用

1. **I 加到“微微超调”**
   让它微微振荡，别压平
2. **加 P**
   抑制振荡，同时加快响应
3. **不加 D**
   二阶差分吃噪声，看不出明显效果
4. **每次只改一个量**
   看上升、超调、受阻恢复

顺序不是定理

先 P 后 I 和先 I 后 P 都能到同一组系数；先给 I 会显得慢，是调法，不是算法。

记录参数

$K_p/K_i/K_d$、T_s、限幅、负载一起写进注释，不然下周就忘了。

先定数量级

Kp/Ki 从 0.1、1 试起，别一上来给 10

一次一个量

改两个说不清是谁的功劳

看曲线

上升、超调、受阻恢复

实际项目里，内环外环各用什么

同样是 PID，内环盯执行器，外环盯目标；写法跟着场景走。

场景	用什么	为什么
电机速度环 本次	增量 PI	编码器计数有量化噪声，占空比要限幅，受阻后要立刻补
平衡车最内角速度环	增量 PI	内环周期短、输出限幅、噪声大，PI 够用
无人机最内角速度环	位置 PID	内环要快、要直接限幅，D 压超调
外环（角度 / 位置）	位置 PD	只给内环目标，不需要再积一次；D 提供阻尼

外环
位置 PD
角度/位置偏差 → 给内环一个目标速度

→

内环
增量 PI（或位置 PID）
速度偏差 → 给执行器一个占空比

→

执行器
限幅输出
电机 / 舵机 / 电调，u 有上限

规律

内环输出直接给执行器，偏增量 PI；外环只给内环目标，偏位置 PD。没有一条规则套所有——看执行器、看噪声、看要不要抗饱和。

这次考什么，怎样算通过

严格按企划书这五条，现场逐条对。

① 上电
静止
y≈0，u 不抖

② 稳定
跟到目标
±2 个计数以内

③ 挡住
输出补上
u 上升，y 平住

④ 松手
回到目标
不来回摆

⑤ 曲线
三条线同屏
目标 / 实际 / 占空比

项	标准	常见失分
算法	直流有刷电机速度闭环，用增量式 PID	写成位置式，或把 rpm 和 count 混用
目标	目标转速以编码器脉冲计数给出：每 5 ms 采样周期 50 个脉冲	目标、反馈量纲不一致
表现	受阻时控制输出明显增大、转速明显维持	只看到 u 变大、转速掉一半
验收	现场演示，负责人当场验收	演示时讲不清
时长	1 周	拖到第二周

VOFA 上怎么核对

把目标、实际、占空比三条线同时发上去：挡住时占空比明显上升、转速基本平住，就是闭环生效的证据。演示顺序：上电 → 稳定 → 挡住 → 松手 → 看曲线。

判定只看这五条

工程分模块、现场讲代码是培训要求，不单独作为通过条件。

烧录时不上电，烧完再上电

烧录全程电池断开、由下载器供电；确认程序能跑，再断电接电池。

1 **断开一切电源**

开关拨 OFF，电池插头不插。

2 **接下载器**

3V3 / SWDIO / SWCLK / GND，以丝印为准。

3 **只插 USB**

由下载器给板子供电，电池仍断开。

4 **编译**

MounRiver 里 Ctrl+B，确认 0 error。

5 **下载**

点 Download，等 Success。

6 **复位观察**

串口有打印，VOFA 里能看到 y 和 u。

7 **断电接电池**

先拔 USB，再插电池，最后开开关。

8 **架空试转**

轮子离地，确认方向与转速正常。

9 **落地演示**

确认无误再放地面，别一开始就带载。

轮子必须架空

下载后电机可能立刻转起来，落地会窜出去。

不要带电插拔电机线

拔插瞬间的电感反峰会打坏驱动。

带电池能不能烧

可以，但电流突变可能让板子复位；仍建议先用 USB 供电烧录。

找不到芯片

先看下载器驱动和接线，再确认板子由 USB 供电。

下载成功但不转

先看串口有没有打印，再看 PWM 引脚和方向引脚。

一上电就窜

十有八九是方向反了或目标太大，架空再试。

收个尾

记住三句话

- **没有反馈就没有闭环**　编码器把“转了多少”变成可以比较的数字。
- **周期必须固定**　5 ms 里做完读、算、写，最好放进定时器中断。
- **顺序看写法**　位置式 P → I → D；增量式 I → P，D 基本不用。

软件培训 · 第三次　|　电机闭环控制　|　演示数据为教学仿真，实际以实物为准

下一讲 · 第四次培训：嵌入式数字图像处理入门